Digital Image Processing

Assignment One REPORT

Connected Component Labeling

Student names:

Hala Bani Munieh

Sawsan Dababseh

Huda Souf

Supervisor:

Dr. Fadi Dridi

Date:

12/4/2026

Introduction

In this assignment, we implemented connected component labeling using MATLAB. The main objective was to understand how pixels in an image can be grouped into regions based on their connectivity.

We worked on four main tasks:

- 4-connectivity labeling
- 8-connectivity labeling
- Intensity range filtering
- Size filtering

All algorithms were implemented manually without using built-in MATLAB functions for labeling.

Input and Preprocessing

At the beginning of the program, the user is asked to enter:

- The image file name
- A threshold value for binarization
- Minimum and maximum intensity values
- Minimum size for filtering

After that:

- The image is read
- If it is a color image, it is converted to grayscale
- A binary image is created using the threshold value

Pixels greater than the threshold are considered foreground (1), while others are background (0).

Methodology

1. 4-Connectivity Labeling

In this method, each pixel is connected only to:

- The left neighbor
- The top neighbor

If a foreground pixel has no labeled neighbors, a new label is assigned.
If it has labeled neighbors, the smallest label is assigned, and equivalences between labels are resolved using a parent array.

After scanning the image, a second pass is performed to ensure all equivalent labels are unified.

2. 8-Connectivity Labeling

In this method, each pixel is connected to:

- Left
- Top
- Top-left
- Top-right

This increases the connectivity between pixels, resulting in fewer and larger connected components compared to 4-connectivity.

3. Intensity Range-Based Labeling

In this step, we define a range of intensity values:

V=[min,max]

Only pixels whose intensity values fall within this range are kept. All other pixels are removed from the labeled image.

After selecting the intensity range, the labeling algorithm is applied again on the filtered pixels to identify connected components within this range.

4. Size Filtering

In this step, small connected components are removed based on their size.

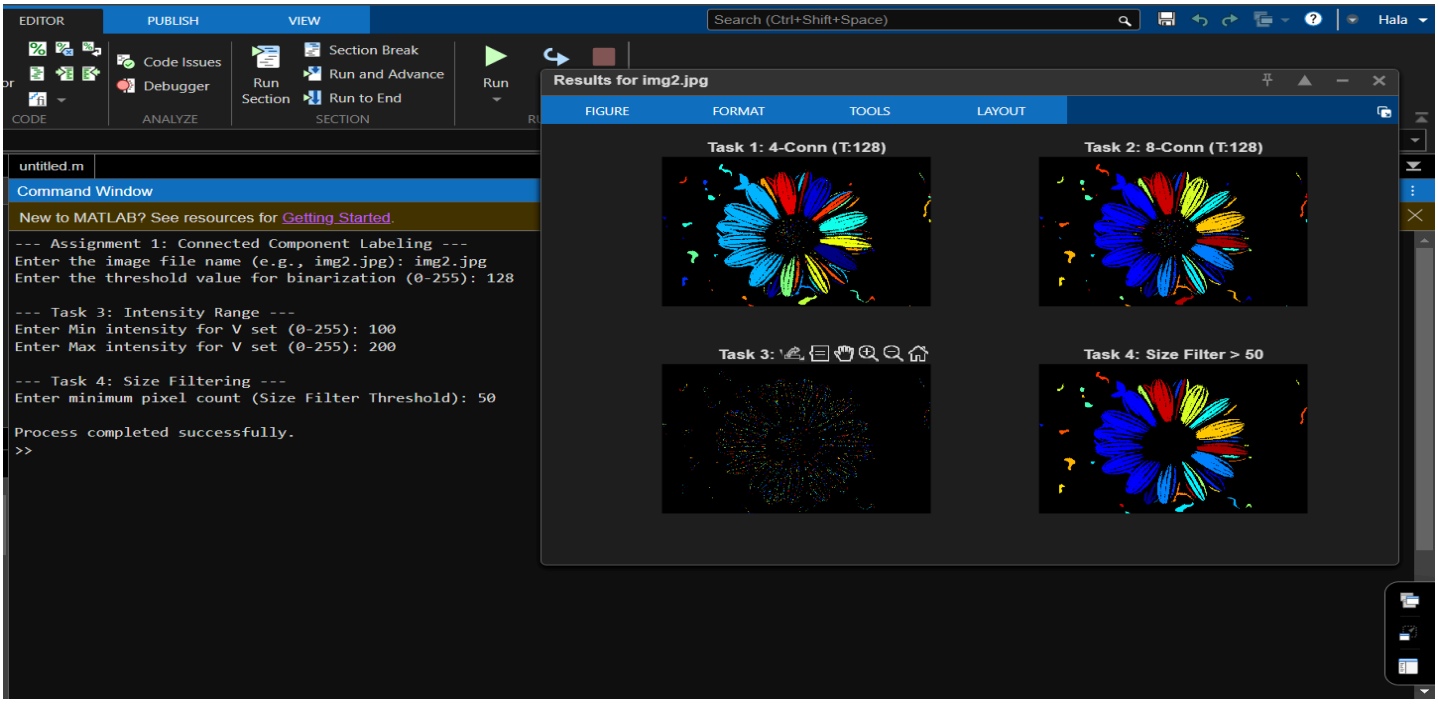
For each labeled region:

- The number of pixels is calculated
- If the size is smaller than the given threshold, the region is removed

This helps eliminate noise and keep only significant objects.

Results and Discussion

1. Results for Image 1 (img2.jpg)

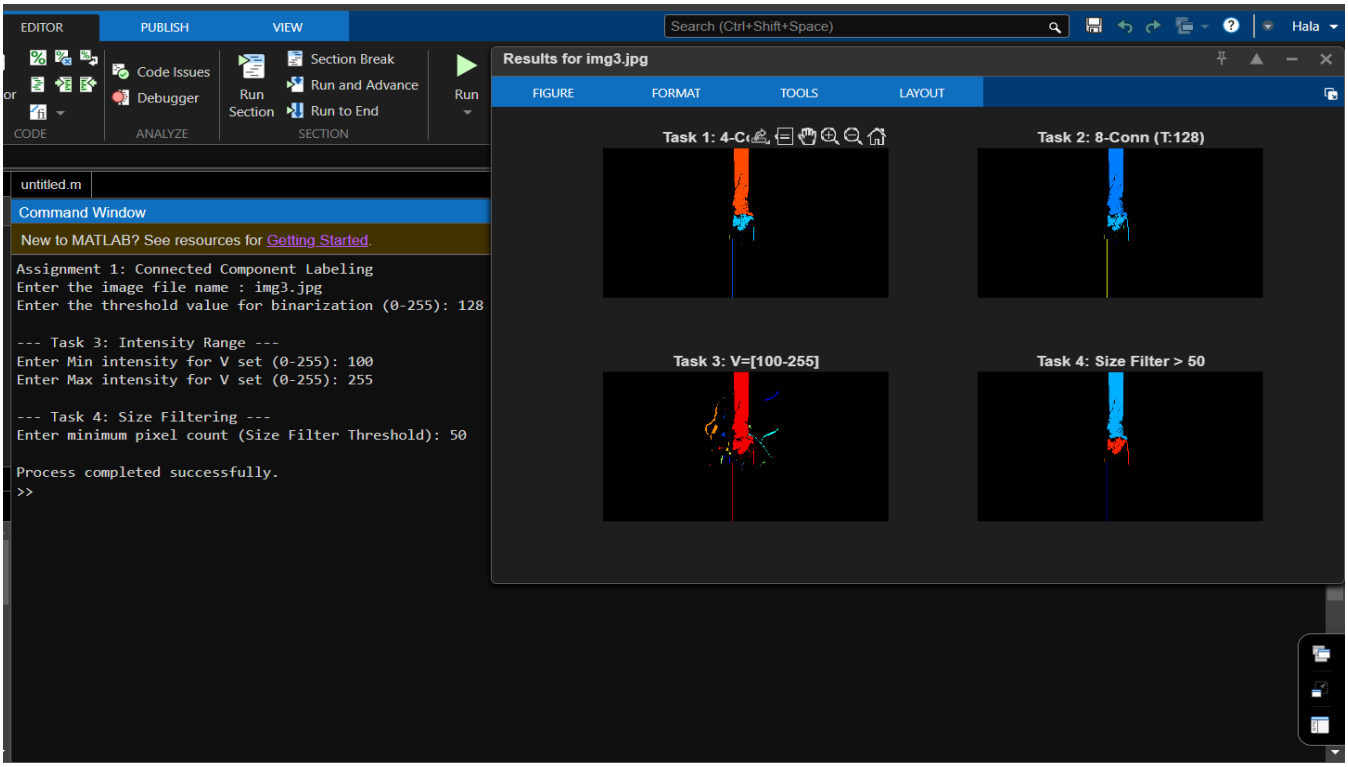


Analysis & Observation:

In this image, I performed the segmentation and labeling on a high-contrast binary-like flower image.

- **Task 1 & 2 (4-Conn vs 8-Conn):** I noticed that using 8-connectivity (Task 2) resulted in more stable components. In 4-connectivity, some petals appeared fragmented, while 8-connectivity joined pixels touching at corners, representing the petals as whole objects.
- **Task 3 (Intensity Range):** I set the range to V=[100-200]. Since the image is mostly black and white, this range only captured the "grayish" edges and noisy pixels. The main body of the flower disappeared because its intensity was either above 200 (white) or below 100 (black).
- **Task 4 (Size Filter):** By setting the threshold to 50 pixels, the code successfully removed the small white "noise" dots scattered around the flower. Only the large petals remained, which proves that the area-based filtering logic is working correctly.

2. Results for Image 2 (img3.jpg)



Analysis & Observation:

This image was interesting because it contains distinct colors (Blue and Red) which have different intensity values when converted to grayscale.

- **Task 1 & 2:** The labeling algorithm assigned different colors (labels) to the blue mask parts and the red dripping parts. The connectivity logic treated them as separate components based on the threshold 128 I provided.
- **Task 3 (Intensity Range):** I used a wider range V=[100-255]. This captured the brighter parts of the illustration (the red and bright blue areas). The dark parts of the cape were excluded because their pixel values were below 100. This confirms that the modification of the algorithm to check intensity values directly instead of a binary mask is accurate.
- **Task 4 (Size Filter):** I used a threshold of 50. You can see in the result that the very tiny droplets and thin lines were eliminated, leaving only the main head structure and the central red light.

Observations

From the results, we can observe that:

- 8-connectivity results in fewer connected components than 4-connectivity
- The threshold value strongly affects the binary image
- Intensity range filtering helps isolate specific parts of the image
- Size filtering is effective in removing noise
- The visual results clearly demonstrate how connectivity type and parameter selection influence the segmentation quality and the number of detected components

Conclusion

In this assignment, we successfully implemented connected component labeling using different techniques.

This assignment helped us understand how:

- Pixels are grouped based on connectivity
- Different parameters affect the output
- Filtering techniques improve the final result

Overall, implementing the algorithm manually gave us a deeper understanding of how image segmentation works.